Reflective Journal:

Lab 05 – Data Preparation

Course: ITAI-1371 Machine Learning

Author: Arinze, Ajmal, Johna

Assignment: Module 05 Lab – Data Preparation

Author : Arinze, Ajmal, Johna

Arinze Okafor

1. Executive Summary & Core Objective

The goal of this lab was to clean and prepare tabular data from the Titanic dataset for machine learning. Preprocessing isn't just routine cleanup—it directly affects whether a model converges properly or picks up invalid patterns. This lab focused on three core steps: imputing missing values with the median, converting categorical variables using one-hot encoding, and standardizing numerical features using z-scores.

2. Implementation Choices & Trade-offs

Imputation: Median vs. Mean

Running df.isnull().sum() highlighted missing values across Cabin, Embarked, and notably 177 missing rows in Age. Dropping those rows wasn't practical because losing nearly 20% of the data would discard valuable signal from features like Pclass and Fare.

Rather than filling missing entries with the mean—which is easily skewed upward by older passengers—I used median imputation via df['Age'].fillna(df['Age'].median()). The median gave a more stable central value that wasn't distorted by right-tail outliers.

Categorical Encoding: One-Hot Encoding

Models require numeric inputs, but using standard label encoding on nominal fields like Sex and Embarked would imply a false ranking (e.g., treating category 2 as greater than category 1).

Using pd.get_dummies() converted these categories into separate binary indicator columns. Including drop_first=True was essential to remove the baseline category, preventing multicollinearity (the dummy variable trap) and keeping feature columns linearly independent for linear and logistic models.

Feature Scaling: Standardization with StandardScaler

Looking at Age (roughly 0–80) and Fare (roughly 0–512+) showed a clear scale mismatch. Without scaling, gradient-based optimizers and distance-reliant models like KNN or SVMs would give disproportionate weight to Fare simply because of its larger range.

Using StandardScaler centered the continuous features to a mean of 0 and a standard deviation of 1. Importantly, scaling was applied only to continuous columns (Age, Fare), leaving the one-hot binary columns as 0s and 1s to maintain their direct boolean meaning.

3. Model Sensitivity to Preprocessing

How much preprocessing matters depends entirely on the downstream model family:

- **Distance & Gradient-Based Models:** Algorithms like Logistic Regression, SVMs, and neural networks require scaled inputs. Unscaled features warp the loss surface, leading to erratic gradient steps and slow convergence.
- **Tree-Based Models:** Decision Trees and Random Forests split on individual features one at a time ($x_i > c$). Because they evaluate order rather than vector distances, monotonic transformations like standardization have no effect on split points or predictions.

Preprocessing pipelines must be tailored to the specific model type being deployed.

4. Key Takeaways

Writing the code for fillna(), get_dummies(), and StandardScaler() is straightforward, but understanding the math and assumptions behind each function determines whether the data pipeline holds up in production.

Immediate Next Steps:

1. Bundle these preprocessing steps into a reusable sklearn.pipeline.Pipeline with ColumnTransformer to prevent data leakage during train/test splits.
2. Evaluate downstream classifier performance across both tree models (Random Forest) and linear/distance models (Logistic Regression, KNN) using the transformed dataset.

John'a Matthews

Reflective Journal

John'a Matthews

In this lab, I learned about the importance of data preparation in machine learning and how raw data needs to be cleaned and organized before it can be used to train a model. Working with the Titanic dataset helped me understand how missing values, categorical data, and numerical features can affect the machine learning process. I learned that data preparation is an important step because the quality of the data can influence how well a model performs.

One of the main concepts I learned was handling missing values through imputation. The Age column in the Titanic dataset contained many missing values, so I used the median to fill them in. I learned that the median is often a better choice than the mean when data contains outliers or is skewed because it is less affected by extremely high or low values. This helped me understand that choosing the right method for handling missing data depends on the type and distribution of the data. I also learned that filling missing values allows us to keep more information instead of deleting rows that contain incomplete data.

Another important concept I learned was one-hot encoding. Machine learning models often need numerical data instead of text, so I used pd.get_dummies() to convert the Sex and Embarked columns into numerical columns. This helped me understand how categorical information can be represented using zeros and ones. I also learned that setting drop_first=True removes one category from each encoded feature to help avoid multicollinearity in certain models.

I also learned about feature scaling and why it is useful for numerical data. I used StandardScaler to scale the Age and Fare columns, so their values would have a mean of approximately zero and a standard deviation of approximately one. This showed me how

features with different ranges can be placed on a more comparable scale. However, I learned that scaling is not usually necessary for decision tree models because they make decisions by splitting data based on feature values rather than on the values themselves.

Overall, this lab helped me connect data preparation techniques to the larger machine learning workflow. I now understand that missing values must be handled, categorical features may need to be encoded, and numerical features may need to be scaled depending on the model being used. I also learned that there is not always one correct preprocessing method for every dataset. The best approach depends on the data and the machine learning algorithm. These skills will help me prepare datasets more effectively and understand the steps required before building a machine learning model.

Hamza Ajmal

Module 05 Lab – Reflective Journal

ITAI-1371: Introduction to Machine Learning | Data Preparation

Understanding the purpose of data preparation

This lab helped me understand that preparing data is an important part of a machine-learning workflow, not just a step to make a table look cleaner. Real datasets can contain missing values and text categories, and many machinelearning algorithms need consistent numerical inputs. The preparation choices also affect what information is kept and how a model can interpret the features.

Handling missing values

In the Titanic dataset, the initial inspection showed 177 missing values in Age, 687 in Cabin, and 2 in Embarked. For the assigned task, I calculated the median of Age and used it to fill the missing Age entries. The verification output showed zero missing values remaining in Age. I learned why the median can be a sensible choice: it is less affected by unusually high or low observations than the mean. I also noticed that this task only handled Age, so the missing Cabin and Embarked values remained. This is a useful reminder to check each column rather than assume that one imputation step cleans the entire dataset.

Encoding categorical features

The next task used one-hot encoding for Sex and Embarked. The output contained indicator columns such as Sex_male, Embarked_Q, and Embarked_S. These columns represent categories using zeros and ones, allowing algorithms to process the information without interpreting the original category names as quantities. Setting drop_first=True leaves one category as the reference category, which can avoid redundant indicators in some models.

Feature scaling and model choice

StandardScaler was applied to Age and Fare. In the output, the values changed from their original units to standardized values, which makes the features comparable in scale. This is useful for models that are sensitive to feature magnitude, such as k-nearest neighbors and support vector machines. Scaling is generally less important for decision trees because their splits depend on thresholds and the ordering of feature values.

Workflow lessons and next steps

A key lesson is to execute the notebook in order and verify each output before moving on. When a setup-cell syntax error prevented the dataframe from being created, the later tasks produced NameError messages. Fixing the first error allowed the later steps to run. This showed me how errors early in a notebook can affect every cell that depends on earlier variables. I also learned that a production workflow should split data before fitting preprocessing tools: imputers, encoders, and scalers should be fitted on training data only and then applied to test data, avoiding data leakage. In a future project, I would use a pipeline to keep these steps consistent and easier to reproduce.